

Lab #4, Semester 1, 2026

Submission deadline: **Friday, 17 April 2026, 11:59 PM**

Total Marks: **20 (Value: 7%)**

Submission Guidelines

You should construct a Python 3 program containing your solution to the given problem and submit your program electronically on Moodle. The name of the file containing your code should be your student ID e.g. **12345678.py**. Remember you need to submit the program as a single file and copy-paste the same program in the provided text box. **Please note that this is an individual project.**

- Your program will be automatically run on Moodle for sample test cases provided in the description sheet if you click the “check” link. However, this does not test all required criteria, and your submission will be thoroughly tested for grading purposes after the due date.
- You have only one attempt to submit, so do not submit until you are satisfied with your attempt.
- All open submissions at the time of the deadline will be automatically submitted. Once your attempt is submitted, there is no way in the system to open/reverse/modify it.
- You must submit your solution before the deadline listed above. Following UWA exceptional variation of assessment (EVA) policy, A penalty of 5 per cent of the total mark allocated for the assessment item is deducted per day for the first 7 days (including weekends and public holidays) after which the assignment is not accepted. For the first two days there will be a penalty waiver, which means that if you submit within that 48-hour period, the assessment will be marked late but the late penalty will not be applied. After 48 hours, the accrued penalty will apply, i.e. a 15% deduction will be applied on Day 3 (after 48 hours), with an additional 5% per day after that (to day 7). For more information, please see the [website](#).

You are expected to have read and understood the University's [guidelines on academic conduct](#). In accordance with this policy, you may discuss with other students the general principles required to understand this lab, but the work you submit must be the result of your own effort. Plagiarism detection, and other systems for detecting potential malpractice, will therefore be used. Besides, if what you submit is not your own work then you will have learned little and will therefore, likely, fail the final exam.

Lab Overview

The XYZ research institute has commissioned your services to develop a data analysis tool that can provide insights into the traffic distribution across Western Australia. You will use the data derived from the [Traffic Digest](#) dataset from Main Roads Western Australia. The dataset shows the locations of traffic count sites on selected roads across Western Australia and provides the average number of vehicles recorded. More information can be found on the [website](#) and the [web visualiser](#).

You are required to write a Python 3 program that will read the data in a CSV file and conduct analysis. Note that the data used in this lab is different from the raw data from Traffic Digest, and you should develop your program based on the sample CSV file provided for this lab. The CSV file has the following columns:

- **X**: Longitude of the traffic count site.
- **Y**: Latitude of the traffic count site.
- **SITE_NO**: Identifier for the traffic count site.
- **ROAD_NAME**: Name of the road on which the count site is located.
- **LOCATION_DESC**: Description of the site location, typically relative to a nearby road or landmark.
- **TRAFFIC_YEAR**: The financial year for which the traffic data was collected.
- **MON_SUN**: Daily traffic volume, averaged over all days of the week (Monday–Sunday).
- **MON_FRI**: Daily traffic volume, averaged over weekdays only (Monday–Friday).
- **SAT_SUN**: Daily traffic volume, averaged over weekends only (Saturday–Sunday).
- **PCT_HEAVY_MON_SUN**: Percentage of heavy vehicles in the Monday–Sunday traffic volume.
- **PCT_HEAVY_MON_FRI**: Percentage of heavy vehicles in the Monday–Friday traffic volume.
- **PCT_HEAVY_SAT_SUN**: Percentage of heavy vehicles in the Saturday–Sunday traffic volume.
- **NETWORK_PERFORMANCE_SITE**: Whether the site is a Network Performance Site ("Yes" / "No"). Network Performance Sites are permanently instrumented locations that collect traffic data, as opposed to portable equipment used for short-term sampling at other sites.
- **LG_NO**: Numeric code identifying the Local Government Area (LGA).
- **LG_NAME**: Name of the Local Government Area in which the site is located.
- **RA_NO**: Numeric code identifying the Administrative Region.
- **RA_NAME**: Name of the Administrative Region.
- **OBJECTID**: An internal object identifier for the feature record.
- **GlobalID**: A global identifier for the feature record.

To complete the task, you need to understand these data columns, which involves different terms defined by the data provider, such as "traffic count site", "Network Performance Site", "Local Government Area", and "Administrative Region".

You are required to write a Python 3 program that will read a CSV file. After reading the file, your program is required to complete the following statistical tasks:

- 1) Find the traffic count sites with the highest and lowest traffic volume for a specific Administrative Region, among sites with the traffic data collected in the year 2022/23 or later.
- 2) Calculate the standard deviations for the percentage of heavy vehicles for the traffic count sites within a specific Administrative Region and all the traffic count sites in the dataset.
- 3) Calculate the correlation between the traffic volume on the weekdays and the traffic volume on the weekends, for the traffic count sites that are Network Performance Sites.

Requirements

- 1) You are not allowed to import any external or internal module in python such as `csv`, `math`, `numpy`, etc.
- 2) Ensure your program **does NOT call the `input()` function** at any time. Calling the `input()` function will cause your program to hang, waiting for input that the automated testing system will not provide (in fact, what will happen is that if the marking program detects the call(s), it will not test your code at all which may result in zero grade).
- 3) Your program **should also not call `print()` function** at any time except for the case of graceful termination (if needed). If your program has encountered an error state and is exiting gracefully then your program needs to return `zero` (for number), `None` (for string), or empty list (for list). At no point should you print the program's outputs instead of (or in addition to) returning them or provide a printout of the program's progress in calculating such outputs.
- 4) Do not assume that the input file names will end in `.csv`. File name suffixes such as `.csv` and `.txt` are not mandatory in systems other than Microsoft Windows. Do not enforce that within your program that the file must end with a `.csv` or any other extension (or try to add an extension onto the provided csv file argument), doing so can easily lead to syntax error and losing marks.

Input

Your program must define the function `main` with the following syntax:

```
def main(file_name, ra_name):
```

The input arguments for this function are:

- **IP1** (`file_name`): The name of the CSV file (as string). The first row of the CSV file will contain the headings of the columns. A sample CSV file "SampleData_Lab4.csv" is provided with the description sheet on LMS and Moodle.
- **IP2** (`ra_name`): The name of an Administrative Region **RA_NAME** (as string).

Output

We expect 3 outputs in the order below.

OP1: Output will be a list including two strings.

- The first string is the identifier (**SITE_NO**) for the traffic count site with the highest all-day traffic volume (**MON_SUN**) located within the input Administrative Region (**IP2: ra_name**).
- The second string is the identifier (**SITE_NO**) for the traffic count site with the lowest all-day traffic volume (**MON_SUN**) located within the input Administrative Region (**IP2: ra_name**).

For **OP1**, only traffic count sites with data collected in the year 2022/23 or later should be considered. When there are multiple sites with the same highest/lowest traffic, choose the first one in alphabetical order in terms of the global identifier (**GlobalID**). Hint: Look for `sort()` or `sorted()` function.

OP2: Output will be a list of two numeric values.

- The first element is the standard deviation for the all-day percentage of heavy vehicles (**PCT_HEAVY_MON_SUN**) for the traffic count sites located within the input Administrative Region (**IP2: ra_name**).
- The second element is the standard deviation for the all-day percentage of heavy vehicles (**PCT_HEAVY_MON_SUN**) for all the traffic count sites in the data.

OP3: Output will be a numeric value, which is the correlation between the traffic volume on the weekdays (**MON_FRI**) and the traffic volume on the weekends (**SAT_SUN**). Use all the traffic count sites in the dataset that are Network Performance Sites.

All returned numeric outputs (both in lists and individual) must contain values rounded to **four decimal places** (if required to be rounded off). Do not round the values during calculations. Instead, round them only at the time when you save them into the final output variables.

Examples

Download "SampleData_Lab4.csv" file on LMS or Moodle. An example of how you can call your program from the Python shell and examine the results it returns, is provided below:

```
>> OP1, OP2, OP3 = main("SampleData_Lab4.csv" , "Metropolitan")
```

The returned output variables are:

```
>> OP1
```

```
['5110', '53950']
```

```
>> OP2
```

```
[6.0164, 12.2696]
```

```
>> OP3
```

```
0.9951
```

Assumptions

Your program can assume the following:

- Anything that is meant to be string (e.g., header) will be a string, and anything that is meant to be numeric will be numeric in the CSV file.
- All string data in the CSV files is case-insensitive, which means "Perth" is same as "perth". Your program needs to handle the situation to consider both to be the

same. Similarly, your program needs to handle the string input parameter in the same way.

- The order of columns in each row will follow the order of the headings provided in the first row, and the order of columns will be always the same as in the sample data. However, rows can be in random order except the first row containing the headings.
- No data will be missing in the CSV files; however, values can be zero and must be accounted for mathematical calculations.
[In case any part of the calculation cannot be performed due to zero values or other boundary conditions, do a graceful termination and returning a zero value (for number), `None` (for string) or empty list (for list) depending on the data type. **Your program must not crash.**]
- Number of Administrative Regions will vary, so do not hardcode.
- The `main` function will always be provided with valid input parameters.
- The necessary formulas are provided at the end of this document.

Important grading instruction

Note that you have not been asked to write specific functions. The task has been left to you. However, it is essential that your program defines the top-level function `main(file_name, ra_name)` (hereafter referred to as "`main()`" to save space. Note that when `main()` is written, it still implies that it is defined with its two input arguments). The idea is that within `main()`, the program calls the other functions. Of course, these functions may then call further functions. This is important because when your code is tested on Moodle, the testing program will call your `main()` function. So if you fail to define `main()`, the testing program will not be able to test your code and your submission will be graded zero. Don't forget the submission guidelines provided at the start of this document.

Marking rubric

Your program will be marked out of 20 (later scaled to be out of 7% of the final mark).

16 out of 20 marks will be awarded automatically based on how well your program completes a number of tests, reflecting normal use of the program, and how the program handles various states including, but not limited to, different numbers of rows in the input file and / or any error states. You need to think creatively what your program may face. Your submission will be graded by data files other than the provided sample data file. Therefore, you need to be creative to investigate corner or worst cases. I have provided few guidelines from ACS Accreditation manual at the end of this document which will help you to understand the expectations.

4 out of 20 marks will be awarded for style and efficiency:

- Style (2 marks): Your code should be clear and easy to read. Consider using meaningful variable names, appropriate comments, and including your name and student ID at the top of the program, etc. [Python style guide](#)
- Efficiency (2 marks): Your program should be well-structured and run efficiently.

Style Rubric:

0	Style is poor, gibberish, impossible to understand.
1	Style is fair, with small lapses, such as unclear variable names, little or no commenting, or inconsistent formatting.
2	Excellent style, really easy to read and follow.

Your program will be traversing text files of various sizes (possibly including large csv files), so you need to minimise the number of times your program looks at the same data items.

Efficiency rubric:

0	Code too complicated to judge efficiency or wrong problem tackled.
1	Very poor efficiency, additional loops, inappropriate use of <code>readline()</code> , etc.
2	Good efficiency and works well with large files, etc.

Automated testing is being used so that all submitted programs are being tested the same way. Sometimes it happens that there is one mistake in the program that means that no tests are passed. If the marker can spot the cause and fix it readily, then they are allowed to do that and your - now fixed - program will score whatever it scores from the tests, minus 4 marks per intervention, because other students will not have had the benefit of marker intervention. Still, that's way better than getting zero. On the other hand, if the bug is hard to fix, the marker needs to move on to other submissions.

Extract from Australian Computing Society Accreditation manual 2019:

As per Seoul Accord section D, a complex computing problem will normally have some or all of the following criteria:

- involves wide-ranging or conflicting technical, computing, and other issues;
- has no obvious solution, and requires conceptual thinking and innovative analysis to formulate suitable abstract models;
- a solution requires the use of in-depth computing or domain knowledge and an analytical approach that is based on well-founded principles;
- involves infrequently encountered issues;
- is outside problems encompassed by standards and standard practice for professional computing;
- involves diverse groups of stakeholders with widely varying needs;
- has significant consequences in a range of contexts;
- is a high-level problem possibly including many component parts or sub-problems;
- identification of a requirement or the cause of a problem is ill defined or unknown.

Necessary formulas

i. **Correlation coefficient, r :**

Mathematical formula to calculate correlation is as follows:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

where x_i and y_i are the individual sample points. $\bar{x}$ and $\bar{y}$ are the sample means.

ii. **Standard deviation, s :**

$$s = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N - 1}}$$

where $x_1, x_2, x_3 \dots x_n$ are observed values in the sample data. $\bar{x}$ is the average value of observations and N is the number of observations.